

6. More-Sophisticated Behavior

Example 6.1

```
import java.util.*;

public class Main {

    public static final int MAX_SIZE = 10;    // constant

    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();    // interface + implementation
        list.add(1);        // autoboxing: int → Integer
        list.add(2);

        Set<Integer> set = new HashSet<>(list);    // Set, no duplicates

        Map<Integer, String> map = new HashMap<>();
        map.put(1, "one");
        map.put(2, "two");

        list.forEach(System.out::println); // method reference

        Util.printSize(list); // class method
    }
}

class Util {

    public static void printSize(Collection<?> c) {
        System.out.println(c.size());
    }
}
```

This example shows

- interface vs implementation
- Map, Set
- autoboxing and wrapper classes
- static, final, constant
- main method
- class method
- method reference

Example 6.2

```
/**
 * This class represents a simple counter.
 */
```

```
public class Counter {  
  
    private static int count = 0; // class variable  
  
    public Counter() {  
        count++;  
    }  
  
    public static int getCount() {  
        return count;  
    }  
}
```

6.1 Interface

Definition

An interface defines what methods a class must have.

Features

- No implementation for methods (before Java 8).
- A class **implements** an interface.

Example

```
List<Integer> list = new ArrayList<>();
```

- `List` is an interface.
- `ArrayList` is an implementation.

Remark

- Code should depend on interfaces, not classes.
 - This improves flexibility.
-

6.2 Implementation

Definition

An implementation is a concrete class that provides method bodies.

Features

- Uses `implements` keyword.
- Must implement all abstract methods.

Example

```
class MyList implements List<Integer> { }
```

- This class must define all List methods.

Remark

- One interface can have many implementations.
-

6.3 Map

Definition

A Map stores key–value pairs.

Features

- Keys are unique.
- Values can repeat.

Example

```
Map<Integer, String> map = new HashMap<>();  
map.put(1, "one");
```

- Key: 1
- Value: "one"

Remark

- HashMap has no order.
 - TreeMap is sorted.
-

6.4 Set

Definition

A Set stores elements with no duplicates.

Features

- Each element is unique.
- No index.

Example

```
Set<Integer> set = new HashSet<>();  
set.add(1);  
set.add(1);
```

- Only one `1` is stored.

Remark

- Use Set to remove duplicates.
-

6.5 Autoboxing

Definition

Autoboxing converts primitive types to wrapper objects automatically.

Features

- Happens automatically.
- No cast needed.

Example

```
Integer x = 5;
```

- `int` → `Integer`

Remark

- Unboxing is the reverse process.
-

6.6 Wrapper Classes

Definition

Wrapper classes represent primitive types as objects.

Features

- Needed for collections.
- Immutable objects.

Example

```
Integer a = Integer.valueOf(3);
```

Remark

- Common wrappers: `Integer`, `Double`, `Boolean`.
-

6.7 Javadoc

Definition

Javadoc is a tool for generating documentation from comments.

Features

- Uses `/** ... */`
- Generates HTML docs.

Example

```
/**
 * Returns the size.
 */
public int size() { }
```

Remark

- Very important for APIs and teamwork.
-

6.8 Access Modifier

Definition

Access modifiers control visibility.

Features

- `public`, `protected`, `private`, default.
- Applied to class, field, method.

Example

```
private int value;
```

Remark

- Use the **most restrictive** modifier.
-

6.9 Information Hiding

Definition

Information hiding hides internal details of a class.

Features

- Use `private` fields.
- Access via methods.

Example

```
private int value;

public int getValue() {
    return value;
}
```

Remark

- Prevents misuse.
 - Improves safety.
-

6.10 Class Variable

Definition

A class variable belongs to the class, not objects.

Features

- Declared with `static`.
- Shared by all objects.

Example

```
static int count;
```

Remark

- Also called static variable.
-

6.11 Class Method

Definition

A class method belongs to the class.

Features

- Declared with `static`.
- Cannot access instance variables.

Example

```
public static int getCount() { }
```

Remark

- Call with `ClassName.method()`.
-

6.12 Main Method

Definition

The main method is the program entry point.

Features

- Must be `public static void`.
- Fixed signature.

Example

```
public static void main(String[] args) { }
```

Remark

- JVM starts execution here.
-

6.13 Static

Definition

`static` means belonging to the class.

Features

- Shared memory.
- No object needed.

Example

```
static int x;
```

Remark

- Overuse can reduce flexibility.
-

6.14 Constant

Definition

A constant is a value that cannot change.

Features

- Declared with `static final`.
- Written in uppercase.

Example

```
public static final int MAX = 10;
```

Remark

- Constants improve readability.
-

6.15. Final Keyword

Definition

`final` prevents modification.

Features

- Final variable: cannot change.
- Final method: cannot override.
- Final class: cannot extend.

Example

```
final int x = 5;
```

Remark

- Often used with constants.
-

6.16. Polymorphic Variable

Definition

A polymorphic variable uses a parent type to refer to child objects.

Features

- Interface or superclass type.
- Actual object can vary.

Example

```
List<Integer> list = new ArrayList<>();
```

Remark

- Enables polymorphism.
-

6.17. Method Reference

Definition

A method reference is a shorthand for a lambda.

Features

- Uses `::`
- Cleaner syntax.

Example

```
list.forEach(System.out::println);
```

Remark

- Equivalent to `x -> System.out.println(x)`.